

PING

Beyond the MVP: Voice, Media, Privacy & Sessions

New Feature Areas	Voice notes · Conversation memory · Media uploads & file-safety · Conversation & group controls · 24h Status/Stories · Privacy settings · Invite links · 2FA & multi-device sessions
Object Storage	Cloudflare R2 (S3-compatible, free tier)
Malware Scanning	ClamAV (self-hosted)
OTP / Transactional Email	Brevo (free tier)
Database	MongoDB (unchanged — Atlas M0 for hosting)
Build Plan	11 incremental stages (continuing from the original MVP's Stage 5)

Goal: Extend the Ping MVP into a fuller messaging product — voice, media, privacy controls, and multi-device sessions — while using each feature as a deliberate exercise in a core backend/security concept (safe file handling, TTL data, access control, session design), in preparation for an eventual professional security review.

1. Overview & How This Extends the MVP

This document picks up where the original *Ping — MVP Specification* left off. The MVP (Stages 1–5) delivered 1-on-1 and group real-time messaging with auth, typing indicators, read receipts, and online status. Everything below is additive — it does not change the MVP's core architecture (Spring Boot + STOMP + MongoDB), it extends it.

The one architectural addition that matters most: **media (voice, photos, videos, documents) is never stored in MongoDB.** MongoDB documents cap at 16MB, and stuffing binary blobs into a database built for structured/semi-structured data is the wrong tool for the job regardless of size. Every media feature below follows the same pattern — the file goes to object storage, and MongoDB only ever stores a reference (URL + metadata).

2. New Features — Summary

Voice Notes

Rich, low-bandwidth communication

Storage cost scales with usage

Client records audio (Opus/WebM), uploads to object storage, message carries a reference + duration. Capped recording length (e.g. 2 min) keeps storage predictable.

Conversation Memory

A real point of difference, not another WhatsApp clone

The AI phases are a project of their own — sequence them last

Ping's actual differentiator: a decision, a date, a number mentioned mid-conversation doesn't have to get buried in scroll. A user explicitly saves a message as a memory (category: Important / Event / Financial / Location / Person / Note), stored separately from the conversation so it stays findable. This ships in four phases, only the first two of which are in scope now:

Phase 1 — Manual save. A "Save to Memory" action on any message, category chosen by the user, stored as its own document referencing the source message.

Phase 2 — Memory page. Browse/search/filter saved memories by category and date, jump back to the original message.

Phase 3 — AI suggestion (deferred). Detect likely-important messages and suggest saving them, instead of requiring the user to always initiate it.

Phase 4 — Ask Ping (deferred). Natural-language Q&A over a user's own memories ("When's my meeting with David?").

Phases 3 and 4 need an LLM integration decision (which provider, cost per query, latency budget) that's a whole separate piece of engineering — deliberately sequenced after every non-AI stage below, per the same "get the fundamentals solid first" reasoning that applies everywhere else in this plan.

Media Uploads (Photos, Documents) & File Safety

Core "real app" feature

Largest attack surface added in this phase

Validate real file content (magic bytes, not client-supplied MIME type), re-encode images server-side to strip embedded exploits/EXIF, scan with ClamAV before serving, enforce size limits at every layer, randomize stored filenames.

Conversation & Group Controls

Access-control practice

Touches many existing endpoints

Delete conversation (per-user soft delete, not a shared hard delete), block user (must be checked on every send/create-conversation/profile-view path), admin-only remove-member on groups.

24-Hour Status / Stories

TTL-index learning exercise

A small second content system

Text or video status posts that auto-expire via a MongoDB TTL index — no cron job needed. Includes per-viewer visibility (`hiddenFrom`) and an opt-in reshare flag.

Privacy Settings & Chat Menu

Reciprocal privacy model

Toggle last-seen/read-receipt visibility with WhatsApp's reciprocity rule (hide yours, lose the ability to see others'). The chat header's 3-dot menu (block / clear chat / view shared media) reuses the same "shared media" query pattern as the conversation workspace's Files view.

Invite Links

Low complexity, good early win

Unique per-user invite code, tracked via a `referredBy` field on new signups.

2FA (Email OTP) & Multi-Device Sessions

Real auth-engineering depth

Highest complexity in this phase

6-digit hashed, rate-limited, expiring codes for 2FA. A proper `sessions` collection enabling "linked devices," QR-code web login, and per-device revocation — build this last, after single-device auth is solid.

3. Updated Tech Stack (Additions)

Technology	Purpose
Cloudflare R2 (S3-compatible SDK)	Object storage for voice notes, photos, documents, status media — no egress fees
Apache Tika	True file-type detection by content (magic bytes), not by trusting client MIME type
Thumbnailator / Java ImageIO	Server-side image re-encoding — strips embedded exploits and EXIF metadata
ClamAV (Docker sidecar)	Malware scanning for uploaded files before they're servable
Brevo (SMTP/API)	Transactional email for 2FA one-time codes
Spring Scheduling / Mongo TTL indexes	Automatic expiry of 24h status posts and OTP codes

4. Database Schema Changes

users (fields added)

```
{
  ...existing fields,
  "blockedUsers": ["userId", "..."],
  "privacy": {
    "showLastSeen": true,
    "showReadReceipts": true,
    "statusPrivacy": "EVERYONE | HIDDEN_FROM"
  },
  "hiddenStatusFrom": ["userId", "..."],
  "inviteCode": "string (unique)",
  "referredBy": "userId (nullable)"
}
```

conversations (fields added)

```
{
  ...existing fields,
  "clearedAt": { "userId1": "datetime", "userId2": "datetime" }
  // messages before a user's clearedAt are hidden for that user only
}
```

messages (type + attachment)

```
{
  ...existing fields,
  "type": "TEXT | IMAGE | FILE | VOICE | SYSTEM",
  "attachment": {
    "url": "string",
    "mimeType": "string",
    "sizeBytes": "number",
    "durationSeconds": "number (voice only, nullable)",
    "scanStatus": "PENDING | CLEAN | REJECTED"
  }
}
```

memories (new collection)

```
{
  "_id": "ObjectId",
  "userId": "ObjectId",           // whose memory this is – memories are private, not shared
  "conversationId": "ObjectId",
  "messageId": "ObjectId",       // the original message this was saved from
  "content": "string",           // snapshot of the message text at save time
  "category": "IMPORTANT | EVENT | FINANCIAL | LOCATION | PERSON | NOTE",
  "createdAt": "datetime"
}
```

No `expiresAt` here despite the field appearing in the original sketch — memories are meant to persist until the user deletes them; nothing about "important information you asked to remember" should silently vanish.

statuses (new collection)

```
{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "type": "TEXT | VIDEO",
  "content": "string (nullable)",
  "mediaUrl": "string (nullable)",
  "viewedBy": ["userId", "..."],
  "allowResharing": false,
  "originalStatusId": "ObjectId (nullable, set on reshares)",
  "createdAt": "datetime",
  "expiresAt": "datetime" // TTL index: expireAfterSeconds on this field
}
```

sessions (new collection)

```
{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "deviceId": "string",
  "deviceName": "string",
  "deviceType": "WEB | ANDROID | IOS",
  "refreshTokenHash": "string",
  "createdAt": "datetime",
  "lastActiveAt": "datetime"
}
```

otp_codes (new collection)

```
{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "codeHash": "string",
  "purpose": "LOGIN_2FA",
  "attempts": 0,
  "createdAt": "datetime",
  "expiresAt": "datetime" // TTL index
}
```

5. New REST API Endpoints

Media

Endpoint	Description
POST /api/media/upload	Upload a file (voice/image/document) — validates, quarantines, scans, returns a reference once clean
GET /api/media/{id}	Retrieve a signed, time-limited URL for a stored file

Conversation Memory

Endpoint	Description
POST /api/memories	Save a message as a memory (messageId + category)
GET /api/memories?category=&q=	List/search the current user's memories, optionally filtered
DELETE /api/memories/{id}	Delete a memory (does not touch the original message)

Conversations & Groups

Endpoint	Description
DELETE /api/conversations/{id}/clear	Soft-delete (clear) conversation for the current user only
POST /api/users/{id}/block	Block a user
DELETE /api/users/{id}/block	Unblock a user
DELETE /api/conversations/{id}/participants/{userId}	Admin removes a group member

Status / Stories

Endpoint	Description
POST /api/statuses	Create a status post (text or video)
GET /api/statuses/feed	Get active statuses from non-blocked, non-hidden-from contacts
POST /api/statuses/{id}/view	Mark a status as viewed by the current user
POST /api/statuses/{id}/reshare	Reshare a status (only if allowResharing is true)

Privacy & Settings

Endpoint	Description
----------	-------------

PUT /api/users/me/privacy	Update last-seen / read-receipt / status privacy settings
GET /api/conversations/{id}/media	List shared media in a conversation (3-dot menu "view profile" media grid)

Invite Links

Endpoint	Description
GET /api/users/me/invite-code	Get (or lazily generate) the current user's invite code
POST /api/auth/register?ref={code}	Register via an invite code, sets referredBy

Sessions & 2FA

Endpoint	Description
POST /api/auth/otp/send	Generate + email a 6-digit code (rate-limited)
POST /api/auth/otp/verify	Verify code, issue session tokens
GET /api/sessions	List the current user's active/linked devices
DELETE /api/sessions/{id}	Revoke a device/session
POST /api/sessions/pair	Approve a QR pairing token from an already-authenticated device

6. WebSocket / STOMP Additions

Channel	Direction	Description
/topic/conversation/{id}/system	Server → Client	Group member removed / left notifications
/topic/user/{id}/status-feed	Server → Client	New status posted by a contact
/topic/pairing/{token}	Server → Client	Notifies a waiting web client once its QR pairing token is approved

7. Build Stages (continuing from MVP Stage 5)

Stage 6 — Voice Notes

Goal: users can record, send, and play back short voice messages.

- Object storage bucket + signed-upload flow (Cloudflare R2)
- `Message.attachment` field wired end-to-end
- Client-side recording (MediaRecorder API), max-duration cap
- Playback UI in `MessageBubble`

Stage 7 — Conversation Memory: Manual Save & Memory Page

Goal: save a message as a memory, browse/search saved memories. Building now.

- `memories` collection, `MemoryController` (save / list+search / delete)
- "Save to Memory" action added to the existing message action menu, with a category picker
- A Memories page: grouped by category, jump back to the source message/conversation
- No AI involved yet — Phases 3–4 (AI suggestion, natural-language "Ask Ping") are deferred to the end of this plan on purpose

Stage 8 — Media Upload & File Safety

Goal: photos/documents can be shared without becoming an attack vector.

- Magic-byte validation (Apache Tika), server-side image re-encoding
- ClamAV sidecar + quarantine → scan → promote pipeline
- Size limits enforced client + server side
- Files panel becomes real (already scaffolded in the frontend workspace)

Stage 9 — Conversation & Group Controls

Goal: delete conversation, block user, admin-remove group member.

- Per-user `clearedAt` soft-delete
- `blockedUsers` checks added to send/create-conversation/profile paths
- Admin-only remove-member endpoint + system message broadcast

Stage 10 — 24-Hour Status / Stories

Goal: text/video statuses that auto-expire.

- `statuses` collection + TTL index
- Status feed query respecting `blockedUsers` / `hiddenStatusFrom`
- Reshare flow respecting `allowResharing`

Stage 11 — Privacy Settings & Chat Menu

Goal: last-seen/read-receipt toggles, 3-dot chat menu.

- Reciprocal privacy enforcement in read-receipt/status broadcast logic
- Chat header dropdown: block / clear chat / view shared media

Stage 12 — Invite Links

Goal: users can invite others via a personal link.

- Invite code generation + registration attribution

Stage 13 — 2FA (Email OTP)

Goal: optional 6-digit email verification on login.

- Brevo integration, hashed + rate-limited + expiring codes
- `otp_codes` collection with TTL index

Stage 14 — Multi-Device Sessions, QR Login & PWA

Goal: linked devices, QR web login, installable web app.

- `sessions` collection, device list + revocation UI
- QR pairing flow over a dedicated WebSocket channel
- PWA manifest + service worker ("Download app" → install prompt)

Stage 15 — Conversation Memory: AI Suggestion (Phase 3)

Goal: proactively suggest saving likely-important messages. Deferred — needs an LLM provider decision first.

- Lightweight classification pass on incoming messages (date/amount/decision-shaped text)
- Inline "🧠 Important information detected — Save to Memory?" prompt, never auto-saves without confirmation

Stage 16 — Ask Ping (Memory Phase 4)

Goal: natural-language Q&A over a user's own memories. Build last.

- Query a user's own `memories` (never another user's — this is a per-user, access-controlled search, not a general chat-log AI)
- Retrieval over saved memories + an LLM to phrase the answer, not a raw keyword search

8. Updated Architecture

```
Next.js App (Frontend)
|
|-- REST API (auth, users, conversations, media, statuses, sessions)
|-- WebSocket/STOMP (messages, typing, status, receipts, pairing)
v
Spring Boot Backend
|
|-- MongoDB Atlas ..... users, conversations, messages, memories, statuses, sessions, otp_codes
|-- Cloudflare R2 ..... voice notes, photos, documents, status media
|-- ClamAV (sidecar) ..... malware scan before a file is servable
|-- Brevo ..... 2FA email delivery
```

9. Free-Tier Hosting Recap

Piece	Pick	Note
Database	MongoDB Atlas M0	Free forever, 512MB — media stays out of it
Media storage	Cloudflare R2	10GB free, no egress fees
Malware scan	ClamAV (self-hosted)	Free, private, unlimited
OTP email	Brevo	300/day free forever
Frontend host	Vercel	Effortless for Next.js
Backend host	Render or Fly.io	Free tiers sleep on idle — a real constraint for persistent WebSocket chat, plan around it

This is a continuation of the original MVP Specification document. Sections 1–10 of that document (project overview, MVP schema, endpoints, Stages 1–5, project structure) remain the source of truth for everything already built.